GALGOTIAS UNIVERSITY

Corporate Central and Industrial Outreach

Advisory (Not Compulsory) for Execution of Software Engineer

Newtonite Software Engineering Challenge — Operations Under Pressure: Student Execution Plan

Oct 16, 2026 · @Dr. Vishwa Ratna Mishra

At a glance

Submit by **Monday, 19 October 2026, 10:00 AM** through the Google Form. Late submissions will not be forwarded to Newtonite.

Item	Detail
Employer	Newtonite
Role track	Software Engineering (full-stack)
Challenge	"Operations Under Pressure" — an internal web app to create, assign, track and resolve operational work items across teams
Effort expected by Newtonite	One working day of build (about 8–10 focused hours)
Time available	Friday evening to Monday 10 AM; spread the work, do not stretch the scope
Must submit	Public GitHub repo with source code, README (run + setup), ENGINEERING_DECISIONS.md, automated tests, known limitations
After submission	Live demo and discussion of your architecture and decisions
Work type	Individual; AI coding tools allowed, but you must be able to explain every line

What the brief is really testing

Newtonite is assessing engineering judgement, not feature count. The brief says it plainly: a smaller system with well-considered behaviour beats a large, half-finished one.

- **Product thinking:** the brief leaves decisions open on purpose. You must decide what a work item is, its states, and its roles, and write down your assumptions.
- **Correctness under concurrency:** two people claiming the same item, stale edits, rapid updates, and double-clicked or retried requests.
- **Real authorization:** permissions enforced on the server, per team and per resource, not just hidden buttons.
- **History and accountability:** every important change (owner, priority, state) recorded and visible.
- **Scale awareness:** tens of thousands of items and a growing history, so pagination, indexes and server-side filtering are expected.
- **Ownership:** you will be asked to explain, defend or modify any part of your code live.

Recommended scope and stack

Build one tight vertical slice: work items with a clear workflow, team-based roles, an activity history, and a filtered dashboard. Use the stack you know best; the table is a suggestion, not a requirement.

Minimum viable scope

- **Users and teams:** seeded users (no sign-up flow needed); simple login or token auth; users belong to one or more teams with a role per team (e.g. Viewer, Member, Lead, Admin).
- **Work item:** title, description, type (incident, customer issue, payment, compliance, task), priority, status, team, assignee, created by, due date, version number, timestamps.
- **Workflow:** e.g. Open → In Progress → Awaiting Approval → Resolved → Closed, with server-enforced allowed transitions and an approval step only a Lead can pass.
- **Activity log:** append-only events for every state, owner and priority change, plus comments.
- **Dashboard:** "Assigned to me", "Unassigned urgent", "Awaiting my approval", with search, filters and paginated lists.

Deliberately skip: email delivery, file attachments, SSO, real-time websockets (polling or refetch is enough), mobile app, fancy styling.

Layer	Suggested choice	Why
Backend	Node.js (Express/NestJS) with TypeScript, or Python (FastAPI/Django)	Fast to build, good testing tools

Layer	Suggested choice	Why
Database	PostgreSQL	Transactions, row locks, constraints, indexes, full-text search
ORM / queries	Prisma, Drizzle, SQLAlchemy or Django ORM	Migrations and type safety
Frontend	React + TanStack Query (or Next.js)	Built-in caching, refetch and optimistic updates
Background jobs	A Postgres-backed job/outbox table, or BullMQ with Redis	Reliable async notifications with retries
Tests	Jest/Vitest + Supertest, or Pytest	API-level tests on critical behaviour
Run	Docker Compose (app + Postgres)	One-command setup for the reviewer

Stepwise execution plan

Follow nine steps from Friday evening to Monday morning. Have a working, committed version by Sunday evening; Monday morning is for final checks and submission only.

Friday 16 Oct — evening (2–3 hrs): Understand and design

1. **Read the brief twice and list decisions (45 min).** Write down every open question (what states exist, who can approve, what happens on a conflict) and your assumption for each. These feed ENGINEERING_DECISIONS.md.
2. **Design the data model and API on paper (1.5 hrs).**
   - Tables: `users`, `teams`, `memberships (user, team, role)`, `work_items (…, version)`, `activity_events`, `comments`, `idempotency_keys`, `jobs/outbox`.
   - Endpoints, e.g. `GET /items?status=&priority=&assignee=&q=&cursor=`, `POST /items`, `PATCH /items/:id`, `POST /items/:id/claim`, `POST /items/:id/transition`, `GET /items/:id/activity`.
   - Sketch the state diagram and a role × action permission matrix.

Saturday 17 Oct (6–7 hrs): Backend core

3. **Project setup (1 hr).** Repo, Docker Compose with Postgres, migrations, seed script with 3–4 teams, ~10 users in different roles and a few thousand sample items (to prove pagination works).

4. **Auth and authorization (1.5 hrs).** Login returning a token; middleware that loads the user's team roles; a single permission-check function used by every endpoint, scoped to the item's team.
5. **Work-item API with critical behaviour (3–4 hrs).** Implement create, list (server-side filter, search, cursor pagination, indexes), detail, update with version check, atomic claim, workflow transitions, idempotency keys, and writing an activity event in the same transaction as every change. See the next section.

Sunday 18 Oct (6–7 hrs): Frontend, async, tests

6. **Frontend (3 hrs).** Login, dashboard with "needs attention" views, filterable list, item detail with activity timeline and comments, actions (claim, transition, edit). Handle errors clearly: show a "this item changed, reload" message on a version conflict; roll back optimistic updates when the server rejects them.
7. **Async processing (1 hr).** Write an outbox/job row inside the same transaction (e.g. "notify new assignee"); a worker processes it with retries, a max-attempt limit and an idempotent handler.
8. **Automated tests (2 hrs).** Cover the dangerous behaviours: concurrent claim (only one wins), stale update rejected, duplicate request with the same key creates one item, forbidden transition rejected, user from another team gets 403.

Sunday night to Monday 19 Oct, 9:30 AM: Document and submit

9. **Documentation and submission (2 hrs).**
   - README: prerequisites, `docker compose up`, seed users and passwords, how to run tests.
   - ENGINEERING_DECISIONS.md with ~5 decisions and trade-offs; Known Limitations section; optional architecture diagram.
   - Clone your repo fresh into a new folder and follow your own README. If it does not run, the reviewer cannot evaluate it.
   - Record a 3–5 minute demo video (recommended) and submit the Google Form by 9:30 AM.

Critical behaviours beyond CRUD

Newtonite requires at least three; aim to implement four or five well, each with a test. Pick the ones you can explain confidently.

Situation	One way to implement	Test to write
Two users claim the same item	Conditional update: `UPDATE work_items SET assignee=? WHERE id=? AND assignee IS NULL`; 0 rows updated → 409 Conflict	Fire two claims in parallel; exactly one succeeds
Editing a stale version	Optimistic locking: client sends `version`; update only `WHERE version = ?`, then increment; mismatch → 409 with the latest item	Update with an old version is rejected and data is unchanged
Repeated request (double-click, retry)	Client sends an `Idempotency-Key` header; server stores key + response and returns the stored result on repeat	Same key sent twice creates one item, same response
Workflow rules	Allowed-transitions map checked on the server; approval step only for Leads of that team	Invalid jump (Open → Closed) and non-Lead approval both rejected
Resource-level authorization	Every query and action scoped to the user's teams and role; checked in one service function	User outside the team gets 403/404 on read and write
Reliable async work	Transactional outbox: job row written with the change; worker retries with backoff, caps attempts, handler is idempotent	Job processed twice produces one notification; failed job retries then marks failed
Optimistic UI reconciliation	Update the UI immediately, roll back and show the server's reason on 409/403	Frontend or integration test of the rollback (optional)

Record every one of these changes as an activity event in the same database transaction, so history can never disagree with the item.

Deliverables checklist

Tick every box before submitting.

Repository

- [ ] Public GitHub repo (or private with reviewer access) named `newtonite-ops-<rollno>`
- [ ] Clean commit history showing progress (not one giant final commit)
- [ ] `docker-compose.yml` or equally simple setup; seed data with demo users for each role
- [ ] No secrets committed; `.env.example` provided

README.md

- [ ] What the app does, in 3–4 lines
- [ ] Prerequisites and exact run commands
- [ ] Demo login credentials per role
- [ ] How to run tests
- [ ] Known limitations

ENGINEERING_DECISIONS.md (about 5 decisions)

- [ ] Each decision states: context, options considered, choice, trade-off
- [ ] Suggested topics: data model and history, concurrency control, authorization model, sync vs async, search/pagination, what you chose not to build

Tests

- [ ] Concurrency, idempotency, authorization and workflow tests from the previous section
- [ ] All tests pass with one command

Optional but valuable

- [ ] Architecture diagram (draw.io, Excalidraw or Mermaid in the README)
- [ ] 3–5 minute demo video link
- [ ] Short "next week I would…" list

Google Form for submission

Create the form titled **"Newtonite Software Engineering Challenge — Submission"** with the fields below, and set it to stop accepting responses at 10:00 AM on Monday 19 October. Link responses to a Google Sheet for sharing with Newtonite.

#	Question	Type	Required
1	Full name	Short answer	Yes
2	Enrolment / roll number	Short answer	Yes

#	Question	Type	Required
3	School and programme (e.g. SCSE — B.Tech CSE)	Dropdown	Yes
4	Year / batch	Dropdown	Yes
5	University email ID	Short answer (email validation)	Yes
6	Mobile number	Short answer (10-digit validation)	Yes
7	GitHub repository link (public or reviewer access given)	Short answer (URL validation)	Yes
8	Live deployment link (Render, Railway, Vercel, etc.)	Short answer (URL)	No
9	Demo video link (3–5 min, Drive/YouTube unlisted)	Short answer (URL)	No
10	Tech stack used (frontend, backend, database)	Short answer	Yes
11	Which critical behaviours did you implement?	Checkboxes (concurrent claim, stale-update handling, idempotency, workflow rules, resource-level authorization, reliable async jobs, optimistic UI reconciliation)	Yes
12	Most important engineering decision you made, in 100 words	Paragraph	Yes
13	GitHub profile / portfolio link	Short answer (URL)	No
14	I confirm this is my individual work, I can explain all of it, and the repo runs using the README	Checkbox	Yes

Final discussion and common mistakes

The discussion carries as much weight as the code. Prepare a short answer to each question Newtonite has listed.

Prepare answers for

- How does the architecture work? (Draw it in 60 seconds: browser → API → Postgres → worker.)

- What happens when things fail? (Database down, worker crashes mid-job, network retry, two users colliding.)
- What assumptions did you make, and which would you check with a product manager first?
- Which part of the system matters most, and why?
- What would change at 10× scale? (Read replicas, partitioning the activity table, a search engine, a dedicated queue, websockets.)
- What would you build next with another week?
- Be ready to modify code live, e.g. "add a new status" or "let Leads reassign items".

Common mistakes to avoid

- Building many screens with no concurrency, idempotency or authorization logic. Plain CRUD will not pass.
- Checking permissions only in the frontend.
- Loading all items into the browser and filtering there; always paginate on the server.
- Updating the item and writing history in separate steps, so a failure leaves them out of sync.
- No tests, or tests that only check "the page loads".
- A README that does not work on a fresh machine.
- Copying AI-generated code you cannot explain. Newtonite will ask.
- Submitting at 9:58 AM. Aim for 9:30 AM.